LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING

XMRig Cryptominer Sample Analysis

Malware Analysis

Classification	TLP:CLEAR
Published	2026-06-18
Version	1.0
Author	Lost Boys Cyber
Report Type	Malware Analysis

Published: 2026-06-18 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

XMRig Cryptominer Sample Analysis

1. Executive Summary

The submitted sample `af421881786a...` is best assessed as a Windows cryptomining payload aligned with XMRig. The family call is not derived from one perfect local string hit; instead, it comes from the totality of evidence: a large PE32+ x64 executable with an unusual section layout, sparse but suspicious runtime behavior, and strong external hash-level corroboration that links the exact SHA256 to XMRig/miner reporting and the mining pool domain `pool.hashvault.pro`.

Local sandbox execution did not surface clean miner network beacons or a reliable process tree, and several observed autorun artefacts could plausibly be guest-environment noise. Because of that, this report explicitly separates observed evidence from reported evidence. Observed evidence establishes that the file is suspicious and operationally relevant. External corroboration raises confidence that the payload is a malicious XMRig-based miner rather than an unrelated packed PE.

For defenders, the main value is in sample blocking, retrospective hunting for miner command-line artefacts or pool references, and validating whether any user-writable autoruns or masquerading paths exist on suspected hosts. Cryptomining often signals unauthorized access rather than a self-contained malware event, so confirmed detections should trigger broader host and credential scoping.

2. Sample Metadata

Field	Value
Family	XMRig
Malware Type	Cryptominer
Platform	Windows x64
Primary Sample SHA256	`af421881786af65cf89b28d2a88d37658625f21f9644cf298c438267c7c92572`
Additional Hashes	SHA1 `b6c4a743c2fa8ed7fca8257d26134806b64ef9e0`; MD5 `5807efef92e20ffe074bbdc141cfbdad`
File Type / Format	PE32+ executable (GUI), AMD64, stripped to external PDB
Delivery Theme	Unknown from the recovered archive alone
First Seen / Reported	2026-06-17 internal submission
Analyst Confidence	Medium
Primary Objective	Unauthorized CPU resource hijacking for mining

3. Source Review & Confidence

Source	Contribution	Confidence
Internal sandbox analysis	Execution context, ATT&CK rollup, and candidate persistence	High
Internal static triage	Exact hashes, section layout, imports, and size	High
External hash/campaign corroboration	XMRig attribution and pool references	Medium

Directly observed evidence: hashes, PE characteristics, suspicious section layout, and sandbox telemetry. High-confidence assessment: the sample is malware and likely a miner. Lower-confidence point: the exact persistence mechanism and live pool activity were not fully reconstructed from local runtime evidence alone.

4. Initial Triage and Delivery Assessment

The file arrived in a password-protected archive and was unpacked for local triage. No installer bundle, document lure, or second-stage script was supplied, so the report centers on the payload file and its likely miner role.

Variant / Artefact	Delivery Role	Notes
`af421881786a...sample`	Primary payload	Large PE64 executable analyzed via sandbox and static triage.
Password-protected `.7z` archive	Transfer wrapper	Safe analyst-handling mechanism, not proof of original intrusion packaging.

5. Static Analysis

5.1. File and Build Characteristics

The sample is a 9,899,008-byte PE32+ Windows GUI executable targeting AMD64. It is stripped yet retains an unusual mix of sections, including very large data/code regions and nonstandard names such as `.% ,R`, `.-dx`, and `. Q4`. This is consistent with packed or customized commodity malware builds where the operator is not optimizing for clean software-distribution traits.

Static Signal	Evidence	Analyst Value
Large miner-like payload size	~9.9 MB PE64 executable	Consistent with statically bundled or heavily packaged miner families.
Unusual section layout	Nonstandard section names and very large executable/data regions	Suggests packing, customization, or bundled runtime content.
Minimal import surface	Small import list centered on loader/runtime helpers	Consistent with a packed loader or runtime-resolved functionality.

5.2. Strings, Imports, and Embedded Artefacts

The import table is thin and loader-centric (`LoadLibraryA`, `GetProcAddress`, `GetModuleHandleA`, `CreateSemaphoreW`, `GetVersion`, `CharUpperBuffW`). That pattern fits a payload that resolves functionality dynamically or includes substantial internal runtime logic. Automated static IOC extraction generated mostly noisy pseudo-domains and malformed paths, so those values were excluded from the curated IOC set.

5.3. Static Configuration Clues

Local static triage did not recover clean pool, wallet, or command-line configuration values. The most operationally useful configuration leads came from external hash-level reporting, which associated the sample with `pool.hashvault.pro` and `45.76.89.70`.

6. Code Analysis

6.1. Reverse-Engineering Method

Local validation used `file`, cryptographic hashes, `rabin2 -I`, `rabin2 -i`, and internal triage bundles. Ghidra MCP was attempted in this session but returned unrelated content, so no decompiler-backed function labeling was added. This leaves the family call somewhat more dependent on corroborating intelligence than the other two reports.

6.2. Functions and Logic of Interest

Function / Routine	Evidence	Why It Matters
Runtime loader behavior	`LoadLibraryA`, `GetProcAddress`, `GetModuleHandleA`	Suggests dynamic resolution common in packed malware and miner wrappers.
Semaphore control	`CreateSemaphoreW` import	May support single-instance gating or worker coordination.
Process exit and path handling	`ExitProcess`, `GetModuleFileNameW`	Compatible with a self-contained payload launched from staged disk locations.

6.3. Anti-Analysis, Evasion, and Obfuscation

The nonstandard sectioning and narrow import table support an obfuscation or packing hypothesis. However, no definitive anti-VM or anti-debug branch was confirmed from the available evidence.

7. Dynamic Analysis

7.1. Execution Overview

The sandbox run yielded modest runtime evidence. The summary reported a user-writable `Run` key value for `OneDrive`, but the surrounding sandbox also contained normal `SecurityHealth` and `VBoxTray` autoruns. Without corroborating file-create or process-tree evidence, the `OneDrive` value is treated as suspicious but not definitive persistence proof.

7.2. Filesystem, Registry, Service, and Task Changes

No clearly malicious dropped files, scheduled tasks, or services were confirmed. The main runtime artefact of note is the candidate `HKCU\Software\Microsoft\Windows\CurrentVersion\Run\OneDrive` value pointing to a user-writable path under `C:\Users\analyst\AppData\Local\Microsoft\OneDrive\OneDrive.exe`.

7.3. Network and Command-and-Control Behaviour

The local sandbox run did not capture clean miner traffic. DNS and TLS activity again largely reflected background Microsoft services. For that reason, the curated IOC set excludes the raw triage pseudo-domains and instead retains only the externally corroborated mining-pool leads.

7.4. Observed Capability Summary

Capability	Supporting Dynamic Evidence
Potential persistence	Suspicious `Run` key value in a user-writable location, though further validation is required.
Network-enabled miner behavior	Sandbox and external reporting indicate network communication, but no clean local pool session was extracted.

8. Process Tree and Execution Chain

No trustworthy process tree was preserved in the available artefacts, so the execution chain remains shallow.

Step	Parent Process	Child / Action	Why It Matters
1	Unverified	Direct execution of submitted sample	Confirms controlled execution only.
2	Unverified	Candidate persistence via user-writable `Run` key	Worth hunting, but not strong enough to declare as proven without host corroboration.

9. Persistence and System Modification

The only candidate persistence artefact is the `OneDrive` autorun pointing to a user-scoped path. This is suspicious because miners and commodity malware commonly masquerade as popular software under user-writable directories, but the sandbox evidence is not complete enough to elevate this to fully confirmed persistence.

Modification Type	Artefact	Evidence	Confidence
Registry Run key	`HKCU\Software\Microsoft\Windows\CurrentVersion\Run\OneDrive`	Sandbox summary captured a user-writable autorun target	Medium

10. Network and Infrastructure Analysis

Stable infrastructure was not cleanly observed during local execution. The most useful infrastructure leads therefore come from external hash-level reporting rather than live beacon reconstruction.

IOC / Infrastructure	Type	Role	Stability / Confidence
`pool.hashvault.pro`	Domain	Reported mining pool infrastructure	Medium
`45.76.89.70`	IP	Reported miner-related network endpoint	Medium

11. Indicators of Compromise

This report intentionally excludes noisy auto-extracted pseudo-domains from `static_triage.json`. Those values do not meet the threshold for direct operational use. The curated indicator set focuses on hashes and the externally corroborated pool infrastructure.

IOC Type	Example / Count	Source	Operational Notes
SHA256	1	Observed	Best exact-match block and hunt key.
SHA1 / MD5	2	Observed	Useful for correlation across older toolsets and malware repositories.
Domain / IP	2	Reported	Use as a pivot, not a standalone blocklist, because the infrastructure was not observed live in the sandbox.

12. MITRE ATT&CK Mapping

Technique	Name	Evidence
T1496	Resource Hijacking	XMRig family attribution and miner objective align to unauthorized resource use.
T1060	Registry Run Keys / Startup Folder	Candidate `OneDrive` autorun in a user-writable path.
T1071	Application Layer Protocol	Sandbox and public reporting indicate

		network communication compatible with mining operations.

13. Detection Engineering

13.1. Detection Logic Summary

Signal	Telemetry Source	Analytic Goal	Tuning Notes
Miner CLI/pool strings	Process and command-line telemetry	Catch miner launches or loaders invoking XMRig-style arguments	Tune for legitimate internal mining or performance tools if they exist.
Suspicious user-scoped `OneDrive` autorun	Registry telemetry	Surface masquerading persistence often used by commodity malware	Validate against legitimate OneDrive deployments.

13.2. Detection Content

Primary detection content is stored in `detections/xmrig.yara` and `detections/xmrig-persistence-sigma.yml`.

13.3. Detection Validation and Blind Spots

The YARA rule is intentionally conservative because local string extraction was noisy and the strongest family evidence came from external corroboration. Registry and hunt content may surface benign OneDrive customizations if the environment has unusual startup scripting.

14. Threat Hunting

14.1. Hunt Hypotheses

- Hosts impacted by this sample will show miner-related command-line fragments such as `stratum`, pool names, or `xmrig` executable references.
- If persistence is present, it will masquerade as common software under user-writable directories and pair with sustained CPU-intensive processes.

14.2. Hunt Queries

See `detections/hunts.kql` and `detections/hunts.spl` for the prepared hunt content.

14.3. Triage Guidance for Hunt Hits

Treat any `xmrig` or `stratum` command-line evidence as a likely compromise lead. For `OneDrive` autorun hits, confirm the backing binary path, signer, parent process lineage, and whether the binary is user-writable before escalating.

15. Triage and Incident Response

Isolate confirmed miner hosts when CPU theft is active or when the endpoint also shows other signs of post-compromise activity. Collect the suspicious binary, persistence keys, scheduler data, and parent-process lineage. Because miners frequently ride on top of prior access, scope for credential theft, lateral movement, and remote-management tooling rather than eradicating only the payload.

16. Validation and Emulation

Validation completed for this report includes sandbox review, local static triage, and external corroboration on the exact hash. No deeper decompiler-based reverse engineering was completed in this session because Ghidra MCP was unavailable.

17. Recommendations

Priority	Owner	Action	Rationale

High	SOC / EDR	Block the exact sample hashes and hunt for `xmrig`, `stratum`, or `hashvault` artefacts across endpoints	These are the most durable leads available.
High	IR	Review any host showing suspicious `OneDrive` autoruns or sustained CPU abuse for broader compromise	Miners often indicate unauthorized access rather than a standalone nuisance.
Medium	IT / Endpoint Engineering	Validate legitimate OneDrive startup paths and alert on deviations into user-writable masquerade paths	Reduces false positives and improves persistence detection.

18. References and Refresh Notes

- Internal sandbox and static triage artefacts listed in `sources.md`
- MalwareBazaar and Tria.ge public references for exact-hash corroboration
- Refresh if richer config extraction, wallet data, or live pool traffic is recovered